

GITHUB COPILOT - DEEP DIVE

Getting Started with GitHub Copilot

- What generative AI actually is, and how it turns into an output
- What's actually happening when Copilot suggests code
- Where Copilot shows up in your day-to-day workflow
- Building something real with design patterns
- Prompting habits, limits, and using it responsibly



What Generative AI Actually Is

- Classification models sort into existing categories — spam filters, ticket triage
- Generative AI creates new content from a description — text, code, on the spot
- Good fit: summarizing or spotting patterns in text you provide
- Poor fit: a source of truth for facts or figures it was never given
- Copilot Chat lives entirely in the generative branch



Classification

Sorts existing information into known buckets.

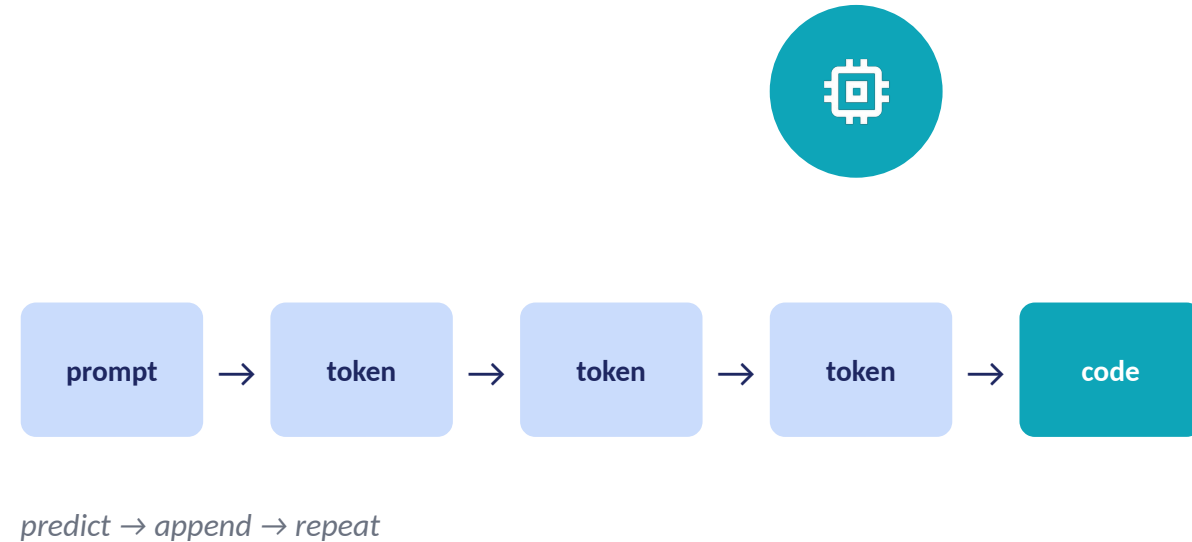


Generative

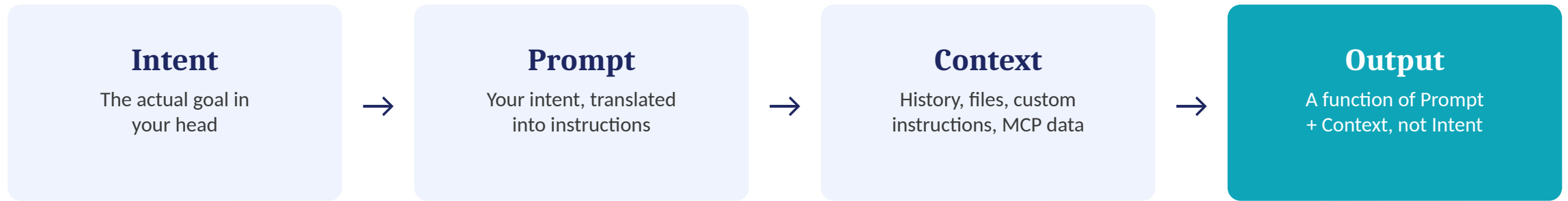
Creates new content that didn't exist a moment ago.

The Mechanism Behind Every Suggestion

- Copilot is a large language model trained on code and natural language
- It predicts one likely next token at a time, then repeats
- A full suggestion is thousands of these predictions chained together
- Fluency comes from having seen the pattern many times, not memorization



From Intent to Output: The Four-Link Chain



WHY THIS MATTERS

A crystal-clear intent paired with a vague prompt and no context still produces a disappointing output — the model can't read minds, only what's actually in front of it.

The Context Window — A Fixed-Size Whiteboard



- The model holds only so much "in view" at once, measured in tokens
- Prompt, attached files, instructions, and the whole conversation share one window
- Once it's full, the oldest content quietly falls out of view — not deleted, just unseen
- A thread that seems to "forget" something said 40 messages back fell off this edge



Fixed-size window — oldest content pushed out first

Problem — Fluent, Not Necessarily Correct

- A plausible-sounding suggestion is not the same as a correct one
- Nothing in the prediction step checks against a compiler, a test, or a fact
- A confidently wrong suggestion is the same mechanism, just a wrong guess
- It looks identical to a confidently right suggestion until you check

```
function handle(input) {  
  const parsed = parse(input);  
  return format(parsed);  
}
```



Correct

```
function handle(input) {  
  const parsed = parse(input);  
  return format(parsed);  
}
```



Confidently Wrong

Same style, same confidence — different outcome.

THE HABIT

Solution — Review Every Suggestion Like a First Draft

- Treat every suggestion as a fast, fluent colleague's first draft, not an oracle
- Review before you accept — every time, not just when something feels off
- This habit repeats in every later module, not just today



WHY THIS MATTERS

This is the single most important habit in the whole course — everything from here on assumes it.

Where You'll Actually Meet Copilot



Inline Completion

Ghost text as you type



Chat & Agent Mode

Synchronous, inside your IDE



Copilot CLI

Terminal-native



Code Review

Automated reviewer on your PRs

Using Inline Completion Well

- Suggestions appear as you type — accept, cycle, or ignore
- Accept in full, or partially, word by word or line by line
- A comment describing intent, ahead of an empty function, steers the first suggestion
- Suggestions range from one line to a full function or class

```
// build a telemetry reading with  
// vehicle id, timestamp, and GPS  
class TelemetryReading {  
    ...  
}
```

A descriptive comment steers the first suggestion.

Building a Real Component with Design Patterns

- Build a small, real piece of the shared practice project
- Naming the pattern and its intent shapes what Copilot produces



Singleton

Exactly one instance — e.g. a shared connection manager



Builder

Several optional fields — e.g. GPS being optional

HANDS-ON Design-Pattern Prompting Comparison

Setting Up the Ask

- In-code exemplars — show the shape you want with an example already in the file
- Instructional scaffolding — break a complex ask into an explicit sequence
- Contextual priming — set role and constraints up front
- Context-added prompting — name or attach the specific file or data involved



Contextual priming, in one prompt

"You're reviewing this for a production ingest service, not a prototype."

Refining and Scoping

- Pattern seeding — start broad, add constraints across follow-up turns
- New thread vs. continuing — a follow-up inherits everything said so far
- Negative prompts — state explicitly what to avoid
- Prompt chaining — split a complex generation into smaller, sequential prompts



A negative prompt, in one line

"Don't use a third-party library for this — no global mutable state."

Knowing Where the Mechanism Runs Out

- Suggestions come from training data, not a live view of your codebase or the world
- Copilot can be confidently wrong about a fact, an API, or a security implication
- Better prompting narrows this risk — it doesn't remove it
- This is why review stays a permanent discipline, not a beginner's caution



The same fluency-vs-correctness mechanism from Slide 6 — revisited here as a permanent limit, not a one-time warning. Module 5 returns to this as a governance topic.

Choosing Your Tool Deliberately



GitHub Copilot

Deep integration across the whole GitHub platform



Amazon Q Developer

AWS-native, tightest fit for AWS-centric workflows



Tabnine

Private/self-hosted deployment, strong for data residency

Feature specifics change fast — verify before presenting a detailed comparison live.

The Responsibility Side of Adopting These Tools



- What you attach or leave open is a data-sensitivity decision, not just a productivity one
- AI output can reflect biases or gaps in its training data
- This is a reason for review discipline, not a reason to avoid the tool

COMING BACK TO THIS

Module 5 covers this formally — this is the first, lightest touch of it.